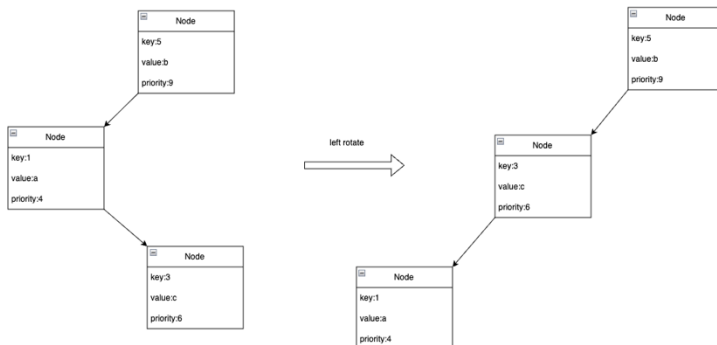
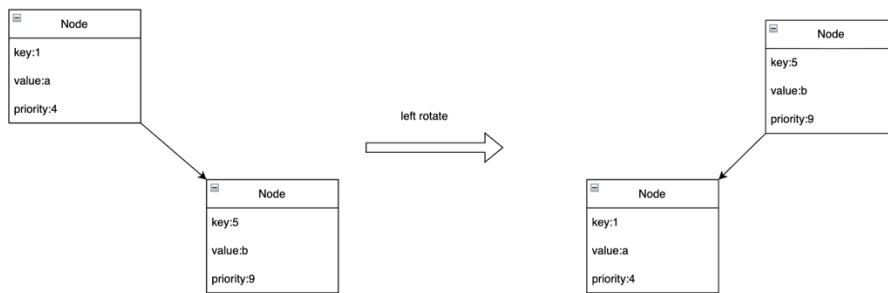
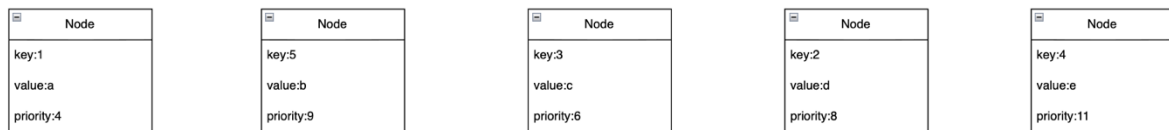
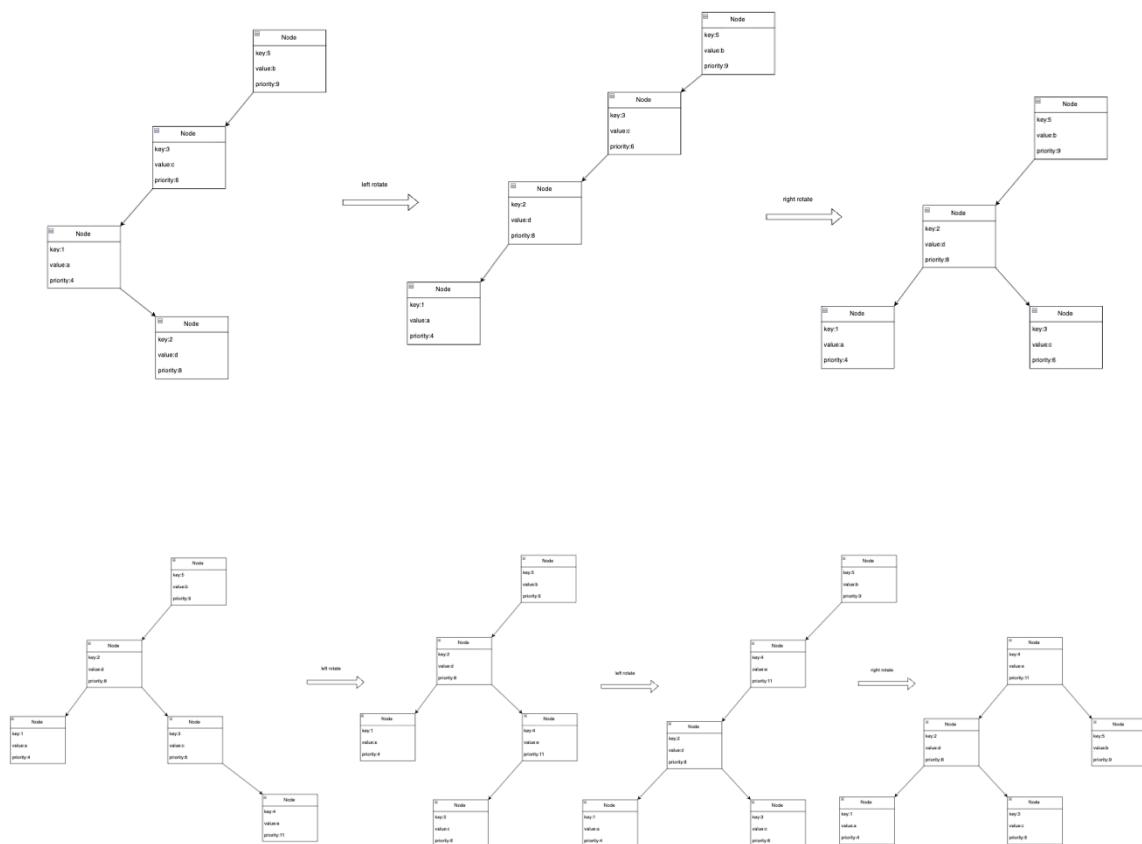


HD2 Report

Worked Example

Each node in the Binary Search Heap (BSH) must satisfy two properties: the keys follow the Binary Search Tree ordering, while the priorities follow a max-heap ordering, where a parent's priority is at least as large as its children's priorities. Each new node is first inserted according to its key and is then rotated upwards while its priority violates the heap property.





Reasoning

A standard Binary Search Tree can become very tall when keys are inserted in an unfavourable order. For example, inserting already sorted keys can create a chain with height n , causing retrieval, insertion and removal to degrade to $O(n)$.

The BSH reduces the influence of insertion order by assigning each node a random priority. After normal BST insertion, rotations move a node with a higher priority above nodes with lower priorities. Therefore, the final structure depends not only on the order of the keys but also on the independently generated priorities.

For a very tall BSH to form, the randomly generated priorities would need to occur in a particular relative order that is compatible with the long chain of keys. As more nodes are added, obtaining such an extreme ordering becomes increasingly unlikely. Instead, high-priority nodes tend to be distributed throughout the key range and are rotated towards the top of the tree, dividing the remaining keys into smaller subtrees.

Consequently, although a worst-case tree of height $O(n)$ is still possible, it is unlikely with random priorities. The expected height is $O(\log n)$, so operations with complexity $O(h)$ are expected to perform efficiently in most cases.

Evidence of Testing

```
/Users/yihaoyu/Desktop/FIT2085/Ontrack_Files/HD2/main
```

```
Root: 4 e 11
```

```
Root left: 2
```

```
Root right: 5
```

```
Root left left: 1
```

```
Root left right: 3
```

```
--- Search Test ---
```

```
Found key 3: c
```

```
Key 100 not found
```

```
--- Remove Non-existing Key Test ---
```

```
Root after remove(100): 4
```

```
Search 100: Not found
```

```
--- Remove Leaf Test ---
```

```
Search 1: Not found
```

```
Root: 4
```

```
Root left: 2
```

```
Root right: 5
```

```
Root left right: 3
```

```
--- Remove One-child Test ---
```

```
Search 2: Not found
```

```
Search 3: Found
```

```
Root: 4
```

```
Root left: 3
```

```
Root right: 5
```

```
Root left parent: 4
```

--- Remove Two-children Root Test ---

Search 4: Not found

Root: 5

Root left: 2

Root left left: 1

Root left right: 3

Root parent is nullptr: Yes

Root left parent: 5

--- Remove Single Root Test ---

Root is nullptr: Yes

Search 10: Not found

--- Remove All Test ---

Tree is empty: Yes

Search 1: Not found

Search 5: Not found

--- Remove Single-left-child Test ---

Search 10: Not found

Search 5: Found

Root: 5

Root parent is nullptr: Yes

